



Title:

Honeypot Deployment and Analysis

Student name and Number: Karnishwar R – 20086868

Module code and Title: B9CY110 COMMUNICATIONS AND NETWORKING SECURITY

Programme and Year: MSc Cybersecurity – 2025-2026

Lecturer: Kingsley Ibomo

Submission Date: 27 April 2026

Word count excluding references and appendices: 3916

Table of Contents

1. Honeypot Lab	3
1.1. Virtual Machine Setup	3
1.2. Software Tools and Technologies	3
1.3. Network Configuration and Isolation	4
1.4. Log Ingestion and SIEM Integration	5
1.5. Honeypot Justification	5
2. Research Based Honeypot Analysis	6
2.1. Methodology	6
2.2. Analysis of Attack Activity	6
2.3. MITRE Attack Mapping	8
2.4. SIEM Visualisation and Detection	8
3. AI Assisted Enrichment	9
3.1. AI Based Privilege Escalation Analysis	9
3.2. AI Based Failed Login Analysis	11
3.3. AI Based Successful Login Analysis	14
3.4. Conclusion of AI Task	17
4. Critical Reflection	17
4.1. Honeypot Limitations	17
4.2. Detection and SIEM Limitations	17
4.3. AI Limitations	17
4.4. Improvements	18
5. Conclusion	18
6. References	18
7. Github and Video Link	19
8. Appendix	20

1. Honeypot Lab:

1.1 Virtual Machine Setup:

An isolated cybersecurity laboratory environment was set up using virtualisation tools to emulate the attacker and target systems within a controlled environment. The lab setup was designed to create a honeypot system that can capture suspicious traffic and generate telemetry to a Security Information and Event Management (SIEM) solution (Singer & Friedman 2014).

The lab environment is made up of two virtual machines in the same private subnet. The honeypot and SIEM systems were installed on Ubuntu and an attacker system on a separate virtual system using Windows. The honeypot system's IP address is displayed in **Figure 1**, and connectivity between the attacker and honeypot system was confirmed using the ICMP ping command, as displayed in **Figure 2**. This demonstrates effective communication between the systems in the isolated network.

The honeypot system had the OpenSSH service installed (OpenSSH, 2026) to serve as the target. The installation of the service is shown in **Figure 3**, and the running status is shown in **Figure 4**. The SSH configuration file was edited to allow controlled attacks **Figure 5**. Password authentication and other settings necessary for testing were enabled.

The honeypot system **Figure 6** was seeded with dummy user accounts such as “admin”, “test” to simulate vulnerabilities in production systems. They were configured with weak passwords to simulate common vulnerabilities in production systems. This would ensure that the honeypot system would be vulnerable to brute force attacks and would generate useful logs for analysis.

1.2 Software Tools and Technologies:

The honeypot was composed of system-level services integrated with Security Information and Event Management (SIEM) tools, to collect, process and correlate security events. The OpenSSH service was used as a low-interaction honeypot (National Institute of Standards and Technology, 2018) to capture interactions and authentications on the system level. This

approach generates real logs from the operating system, not synthetic ones as in emulated honeypots, thus providing a more faithful picture of the attack.

When the attack simulation was conducted, several failed login attempts were recorded in **Figure 7**, which is a sign of a brute force attack (MITRE, 2025). A successful login attempt is shown in **Figure 8**, suggesting that the attacker successfully logged into the system after multiple attempts. The commands entered once the attacker was logged in are illustrated in **Figure 9**, noting the activities of the attacker after successful login.

The honeypot recorded and stored the logs of these activities. The presence of honeypot logs is shown in **Figure 10** and the log file and storage is shown in **Figure 11**. This data is used for further analysis.

The Elastic search was used to implement a SIEM (Elastic, 2025). Elastic search indexed and stored the log data as shown in **Figure 12** for later use with Kibana for visualisation and analysis as shown in **Figure 13**. The Kibana configuration file was edited to connect with Elastic search **Figure 14**, and an enrolment token was created to complete the installation **Figure 15**. The SIEM dashboard deployment is shown in **Figure 16**, confirming that the SIEM was deployed successfully.

1.3 Network Configuration and Isolation:

Under the lab network configuration, the network configuration was made such that there would be total isolation but also communication between the virtual machines. The systems used by the attacker as well as the honeypots were placed on a host only or NAT network with internal IP addresses.

Such a configuration ensures that no other communication outside the lab network took place when conducting the attack experiment. The network was neither connected to the Internet nor connected to any productive system. This is necessary in order to ensure that the standards of ethics and legality are observed when experimenting with cybersecurity issues. Isolated virtual network configuration is also similar to the cybersecurity practice in the real world.

1.4 Log Ingestion and SIEM Integration:

These logs produced from the honeypot were then ingested into the Elastic Stack for analysis purposes. The logs file was uploaded in Kibana through the use of Data Visualiser as seen in **Figure 17**.

With the logs ingested into Kibana, they were now visible in the Discover module in Kibana as illustrated in **Figure 18**. It is here that queries can be performed in order to analyse the logs to determine any patterns such as multiple failed attempts at logging in as well as successful authentications.

Ingesting and analysing logs on a SIEM platform is an important capability to have in modern cybersecurity today (Elastic, 2025).

1.5 Honeypot Justification:

OpenSSH was to build the honeypot rather than existing solutions like Cowrie or T-Pot (Cowrie Project, 2026) because of the objectives defined for the project. The aim of the project was to explore system level logging and SIEM implementation in practice, not build on top of existing infrastructure.

In addition to the fact that OpenSSH runs as a real-life service and creates real-life logs right out of the operating system, this configuration helps capture all relevant information about attacker actions. In particular, this includes events related to authentication attempts, creation of new sessions, execution of various commands, and privilege escalation.

Though technically this honeypot may be regarded as a low-interaction configuration (National Institute of Standards and Technology, 2018), it proves itself to be a good choice for capturing certain parts of the attack lifecycle. More elaborate high-interaction solutions are known to be capable of more thorough logging, but they come with additional challenges.

2. Research Based Honeypot Analysis:

2.1 Methodology:

In the methodology adopted for this project, an attempt was made to create a situation that could be considered an attack through the attacker virtual machine. The attacker performed several SSH logins on the honeypot, which resulted in failures as well as successful logins.

The logs produced by this exercise were then imported into the Elastic search for further analysis. Logs were analysed in Kibana with the help of filters and queries.

Detection rules were written to automatically recognize any suspicious activity such as multiple failed logins, successful logins, and privilege escalation attacks.

2.2 Analysis of Attack Activity:

From the analysis of the logs obtained, it is evident that there was an order of activities following a normal attack pattern.

Firstly, there were several unsuccessful login attempts seen in **Figure 19**, which depict brute-force attacks (MITRE, 2025). The target was the account name “admin”, which took place over a relatively short time span. This implies that there was some form of automation involved during the attack.

Afterwards, there was a successful login attempt recorded in **Figure 20**. From this, can be understood that the attacker managed to break into the system. This marks the point where there was credential compromise (MITRE, 2025).

The attacker then began interacting with the honeypot system after a successful login. This was evidenced by the successful establishment of a session connection in **Figure 22**. This was the post-authentication phase.

Finally, the attacker made use of the sudo command to escalate his privileges, which is visible in **Figure 21**. This means that he was operating the system using root-level permissions (MITRE, 2025).

Correlation of the logs based on time shows a clear and consistent pattern of attack stages. The first thing that stands out is that there were several unsuccessful attempts to login in a very short period of time **Figure 19**, which is indicative of brute-forcing. Then the attack continued with a successful authentication of the account **Figure 20**. Right after that, a session was established **Figure 22**. Finally, there was the privilege escalation through the use of the 'sudo' command **Figure 21**. Based on all of this information, it can be assumed that the attack had a very consistent workflow and was not accidental at all.

This behaviour clearly shows signs of a malicious attack. The continuous attempts to login into the “admin” account show that the attacker was trying out different passwords and usernames to see whether he could get in or not. The next step of successful login means that the attacker was persistent and probably used some tools to automate the process. After logging into the system successfully, he proceeded to execute some commands within the system **Figure 9**. This further proves that the attacker intended to have full control over the system.

One notable point about this experiment is the fact that the IP address of the source is in the range of an internal private network, which is (192.168.64.3). It would be much more problematic if this happened in practice since this might signify an attack being carried out by someone within the organization. Nonetheless, in this particular lab scenario, the internal IP is simply the attacker VM. This serves as one of the main points when dealing with a security analysis task, and context is very important in such a scenario.

The performance of the rules deployed was measured in terms of their capacity to accurately detect suspicious actions. The rule that detected failed login attempts worked properly when alerts were generated after several failures within a specified period of time **Figure 24**. In addition, the rule which detected successful login attempts **Figure 25** identified account compromise incidents. On the other hand, the rule that detected privilege escalation successfully identified usage of the sudo command **Figure 26**. While these three rules performed well in detecting key incidents, they can produce false positives in practical environments due to normal administrative operations triggering an alert.

2.3 MITRE Attack Mapping:

The above mentioned attack pattern can be linked to the MITRE ATTACK methodology (MITRE, 2025) that serves as a framework for analysing attack strategies.

- Initial Access – T1110.001 (Brute Force: Password Guessing)
- Detected by multiple login failures **Figure 19**
- Credential Access/Initial Access – T1078 (Valid Accounts)
- Detected by successful authentication **Figure 20**
- Privilege Escalation – T1548.003 (Sudo Abuse)
- Detected by the exploitation of Sudo command **Figure 21**
- Persistence, Execution - Detected by the presence of sessions and executed commands **Figure 22**

It is clear from the above description that the attack is structured in a very systematic way as opposed to random attack methods.

2.4 SIEM Visualisation and Detection:

With respect to the addition of log data in the Elastic Search, the visualization and detection of the security events were done quite efficiently. The dashboards were prepared which showed the login attempts and the activities within the system as can be seen from **Figure 23**.

The rules in place within the SIEM helped in determining any suspicious behavior and hence, the detection was completed as shown in **Figure 24** for failed login attempts. On the other hand, successful login attempts have been shown in **Figure 25** while privilege escalation alerts have been shown in **Figure 26**. The architecture for the whole system is depicted in **Figure 27**.

3. AI Assisted Enrichment:

3.1 AI Based Privilege Escalation Analysis:

For improving the analysis process of the security incidents, a Gemini AI-based application was adopted for analysing and enhancing the raw logs created by the honeypot deployment. The aim of this process is to recognize the type of suspicious activity, associate it with different threat models, and derive valuable security information from the activity Wang (2024).

1) Input Provided:

Analyse the following Linux security logs.

1. Summarize the activity
2. Identify potential security threats
3. Map to MITRE ATT&CK techniques
4. Extract any Indicators of Compromise (IOCs)

Logs:

Apr 25 22:04:35 ubuntu sudo: session opened for user root(uid=0) by admin(uid=1002)

2) Output Generated:

Based on the log provided, here is the security analysis of the activity:

1. Summary of Activity

The log entry records a ****privilege escalation**** event on a Linux system named "ubuntu." A user with the account name ****admin**** (UID 1002) successfully used the ``sudo`` command to open a session as the ****root**** user (UID 0). This indicates that the administrative user transitioned from a standard (though likely privileged) shell to a superuser shell.

2. Potential Security Threats

While this could be routine maintenance by a legitimate administrator, in a security context, it raises the following concerns:

* **Unauthorized Privilege Escalation:** If the `admin` account was previously compromised via credential theft, the attacker now has full "God-mode" access to the system.

* **Lack of Context:** The log shows the session opened but does not show the specific command executed (e.g., `sudo -i`, `sudo su`, or a specific binary).

* **Insider Threat:** An authorized user might be performing actions outside of their designated scope.

3. MITRE ATT&CK Mapping

The activity aligns with the following tactics and techniques:

Tactics	Technique ID	Technique Name	Description
:---	:---	:---	:---
Privilege Escalation	[T1548.003]	(https://attack.mitre.org/techniques/T1548/003/)	Abuse Elevation Control Mechanism: Sudo Users may leverage the `sudo` binary to execute commands with the privileges of another user.
Persistence	[T1078.003]	(https://attack.mitre.org/techniques/T1078/003/)	Valid Accounts: Local Accounts Attackers may use existing elevated accounts to maintain access to a system.

4. Indicators of Compromise (IOCs)

Because this is a single log of a successful system function, the "indicators" are behavioral rather than traditional file hashes or IPs.

* **User Account:** `admin` (UID: 1002)

* **Target Account:** `root` (UID: 0)

* **Timestamp:** `Apr 25 22:04:35`

* **Host:** `ubuntu`

> **Note:** To determine if this is truly malicious, you should correlate this timestamp with other logs, such as `auth.log` or `.bash\_history`, to see what commands were executed immediately after the session was opened.

3) Human Validation and Critical Evaluation:

Even though the accuracy of the AI-generated analysis was high in terms of determining the nature of the incident, manual verification was still needed to prove the validity and significance of the event. To be more specific, it was impossible for AI to understand whether the performed activity was malicious or just a part of the regular admin's behavior.

To find out the intentions of the attacker, one should examine the system's logs (e.g., auth.log) and command history file (.bash_history). It is one of the biggest problems associated with AI its inability to understand what kind of data it is dealing with since AI operates only with data provided.

3.2 AI Based Failed Login Analysis:

1) Input Provided:

Analyse the following Linux security logs.

1. Summarize the activity
2. Identify potential security threats
3. Map to MITRE ATT&CK techniques
4. Extract any Indicators of Compromise (IOCs)

Logs:

Apr 25 19:00:56 ubuntu sshd[90963]: Failed password for admin from 192.168.64.3 port 55105 ssh2

2) Output Generated:

Based on the log provided, here is the security analysis of the activity:

1. Summary of Activity

The log entry indicates a **failed SSH login attempt** for the user `admin`. The request originated from the IP address `192.168.64.3` via port `55105` using the SSH2 protocol. The `sshd` service rejected the attempt because the password provided was incorrect.

2. Potential Security Threats

Brute-Force Attack: This is a classic indicator of an automated script or a threat actor attempting to guess the password for a common administrative account.

Credential Stuffing: If this IP attempts logins with different usernames, it may be using lists of leaked credentials from other breaches.

Reconnaissance: An attacker may be probing the system to see if the `admin` account exists and if password authentication is enabled.

3. MITRE ATT&CK Mapping

This activity maps to the following tactics and techniques within the MITRE framework:

Tactic	Technique ID	Technique Name	Description
--------	--------------	----------------	-------------

---	---	---	---
-----	-----	-----	-----

Credential Access	[T1110.001](https://attack.mitre.org/techniques/T1110/001/)	Brute Force: Password Guessing	An adversary may attempt to guess passwords for a specific user.
--------------------------	---	--------------------------------	--

Initial Access	[T1078.003](https://attack.mitre.org/techniques/T1078/003/)	Valid Accounts: Local Accounts	The attempt to access a local "admin" account to gain a foothold in the environment.
-----------------------	---	--------------------------------	--

4. Indicators of Compromise (IOCs)

The following specific data points can be used for blocklisting or further investigation:

* **Source IP Address:** `192.168.64.3` (Note: This is a private/internal IP range, suggesting the threat may be coming from within the local network).

* **Target Username:** `admin`

* **Service/Protocol:** `sshd` / `ssh2`

* **Source Port:** `55105`

Recommended Action

Since the source IP is internal (`192.168.64.3`), you should investigate that specific host immediately. It may be a compromised workstation or an unauthorized device on your network. Check if there are multiple failed attempts from this IP in quick succession; if so, consider implementing `fail2ban` or temporary IP blocking.

3) Human Validation and Critical Evaluation:

While the failure to log in was properly recognized as such by the AI generated analysis, human validation would be needed to verify the importance of the event. One failed attempt does not automatically mean that there is any malicious intent behind the actions performed since it can be easily due to an error made by the user. In order to determine whether there actually was any attack involved, it would be necessary to consider the number and frequency of failed attempts, and to see if it could constitute an attempt at a brute-force attack. If there were multiple unsuccessful attempts, it would point to such attacks taking place.

Moreover, the assumption that the activity could be a brute-force attack was not accompanied by confirmation that there were many failed attempts and no successful ones. It is thus evident that there is an important flaw in using AI for analysis, namely its inability to distinguish different types of behaviour based on insufficient data.

3.3 AI Based Successful Login Analysis:

To continue improving the analysis of security incidents, an application that utilizes the Gemini AI framework was employed to analyse the successful SSH authentication record produced by the honeypot. The objective of this exercise was to establish whether the success of a login attempt can point to possible compromise after analysing failed login attempts.

1) Input Provided:

Analyse the following Linux security logs.

1. Summarize the activity
2. Identify potential security threats
3. Map to MITRE ATT&CK techniques
4. Extract any Indicators of Compromise (IOCs)

Logs:

Apr 25 19:01:35 ubuntu sshd[90965]: Accepted password for admin from 192.168.64.3 port 53349 ssh2

2) Output Generated:

This log entry represents a critical escalation following the previous failed login attempt. Here is the analysis:

1. Summary of Activity

The log records a **successful SSH login** for the user **admin** from the source IP **192.168.64.3**. The connection was established over port **53349** using the SSH2 protocol. This indicates that the person or script at that IP address provided the correct password for the `admin` account.

2. Potential Security Threats

* **Account Takeover:** Given the previous failed attempt from the same IP at 19:00:56, this successful login strongly suggests a **successful Brute-Force or Credential Guessing attack**.

* **Lateral Movement:** Since the source IP (`192.168.64.3`) is an internal address, an attacker may have already compromised one device on your network and is now moving laterally to this `ubuntu` server.

* **Unauthorized Access:** If no authorized administrator was logging in at this specific time, the system should be considered **compromised**.

3. MITRE ATT&CK Mapping

Tactic	Technique ID	Technique Name	Description
:---	:---	:---	:---
Initial Access	[T1078.003]	(https://attack.mitre.org/techniques/T1078/003/)	Valid Accounts: Local Accounts The adversary bypassed login security by using valid credentials for a local account.
Lateral Movement	[T1021.004]	(https://attack.mitre.org/techniques/T1021/004/)	Remote Services: SSH The use of SSH to move from one internal host to another.
Credential Access	[T1110]	(https://attack.mitre.org/techniques/T1110/)	Brute Force The success following a prior failure indicates the culmination of a password-guessing attack.

4. Indicators of Compromise (IOCs)

* **Compromised Account:** `admin`

* **Malicious Source IP:** `192.168.64.3` (Internal/Private IP)

* **Target Host:** `ubuntu`

* **Event Timestamp:** `Apr 25 19:01:35`

⚠ Immediate Recommendations

1. ****Isolate the Host:**** Disconnect `192.168.64.3` from the network immediately to prevent further spread.
2. ****Terminate Sessions:**** Kill all active SSH sessions for the `admin` user on the `ubuntu` server.
3. ****Reset Credentials:**** Change the password for the `admin` account across all systems.
4. ****Audit Logs:**** Check `~/ .bash\_history` for the `admin` user and look for subsequent `sudo` activity to see if the attacker gained root access (as seen in your earlier log at 22:04:35).

3) Human Validation and Critical Evaluation:

The AI analysis gave a good review of the incident but it was manually reviewed to check if it was a malicious activity. A login attempt is not always suspicious as it could just be a trusted user authentication.

You want to correlate the event with other logs to verify the AI analysis conclusion. Particularly, look for failed login attempts from the same source IP address before the successful login. This will strengthen the theory that the attacker was trying to conduct a brute-force attack.

Moreover, it is important to investigate whether the login happened according to the regular pattern. Parameters like login time, user behavior, and action after logging in can help detect suspicious behavior. For instance, one might have tried to execute any command or escalate their privileges.

The AI suggested lateral movement, but the assumption has to be validated. The appearance of an internal IP address does not necessarily mean the attack was malicious. The traffic could have been from a trusted user.

3.4 Conclusion of AI Task:

In conclusion, through this assignment, it is evident that artificial intelligence is able to detect suspicious authentications and put them into context using attack patterns. Nevertheless, it is imperative to correlate multiple attacks and use the human approach in order to evaluate if such activity poses any real danger.

4. Critical Reflection:

4.1. Honeypot Limitations:

Even though the honeypot based on OpenSSH was able to record events regarding authentication and privilege escalation, it is classified as a low-interaction honeypot (National Institute of Standards and Technology, 2018). Low interaction implies that the degree of interaction of the attacker is limited because the attacker may soon realize that the setup cannot provide realistic services or complexity. This is not like high interaction honeypots like Cowrie and T-Pot (Cowrie Project, 2026) which give more real life scenarios than low interaction honeypots.

4.2. Detection and SIEM Limitations:

Even though the Elastic Stack (Elastic, 2025) managed to identify suspicious actions by using rules for setting alarms, the problem is that such a solution depends mostly on already defined rules. Such an implementation may cause false alarms in cases when administrative activities of users look just like malicious attacks. Moreover, since there is no behavioral baseline, the system will not be able to differentiate between ordinary and unusual behavior.

4.3. AI Limitations:

The utilization of AI systems in analysing logs generated structured analysis and automatic mapping to threat models like MITRE ATT&CK. But, the AI system lacked contextual understanding, and it could not establish whether the detected actions were malicious or legitimate administrative activities. This highlights the importance of human intervention

because AI functions solely based on input data without considering environmental factors and intentions Wang (2024).

4.4. Improvements:

Future enhancements for this tool will be in the use of a high-interaction honeypot, such as Cowrie, to record more in-depth actions performed by attackers. Another enhancement will be automating log importation via Filebeat, which currently requires manual uploading. Another possibility would be to use machine learning techniques for better anomaly detection.

5. Conclusion:

In conclusion, the project has managed to demonstrate how the honeypot system works and is integrated into a SIEM system for security purposes. The system has captured several phases of the lifecycle of the attacks, including brute forcing, gaining access, and elevation of privileges.

Through this research, it can be seen that monitoring, detecting, and responding to cyber-attacks is a critical aspect of cybersecurity. It also shows how a SIEM system can be useful in detecting and analysing malicious activity. There are several areas where improvement can be made in the future.

6. References:

1. MITRE, 2025. *MITRE ATT&CK Framework*. [Online] Available at: <https://attack.mitre.org> [Accessed 04 2026].
2. Elastic, 2025. *Elastic Stack Documentation*. [Online] Available at: <https://www.elastic.co/docs> [Accessed 04 2026].

3. Elastic, 2025. *Kibana Guide*. [Online]
Available at: <https://www.elastic.co/docs/get-started/the-stack>
[Accessed 04 2026].
4. OpenSSH, 2026. *OpenSSH Manual Pages*. [Online]
Available at: <https://www.openssh.org/manual.html>
[Accessed 04 2026].
5. Cowrie Project, 2026. *Cowrie SSH/Telnet Honeypot*. [Online]
Available at: <https://docs.cowrie.org/en/stable/>
[Accessed 04 2026].
6. National Institute of Standards and Technology (2018) Framework for Improving Critical Infrastructure Cybersecurity, Version 1.1. NIST CSWP 04162018. Gaithersburg, MD: National Institute of Standards and Technology, p. NIST CSWP 04162018. Available at: <https://doi.org/10.6028/NIST.CSWP.04162018>.
7. Wang, Zehan. (2024). Artificial Intelligence in Cybersecurity Threat Detection. International Journal of Computer Science and Information Technology. 4. 203-209. 10.62051/ijcsit.v4n1.24.
8. Singer, Peter & Friedman, Allan. (2014). Cybersecurity and Cyberwar: What Everyone Needs to Know® What Everyone Needs to Know®. 10.1093/wentk/9780199918096.001.0001.

7. Github and Video Link:

Video Link -

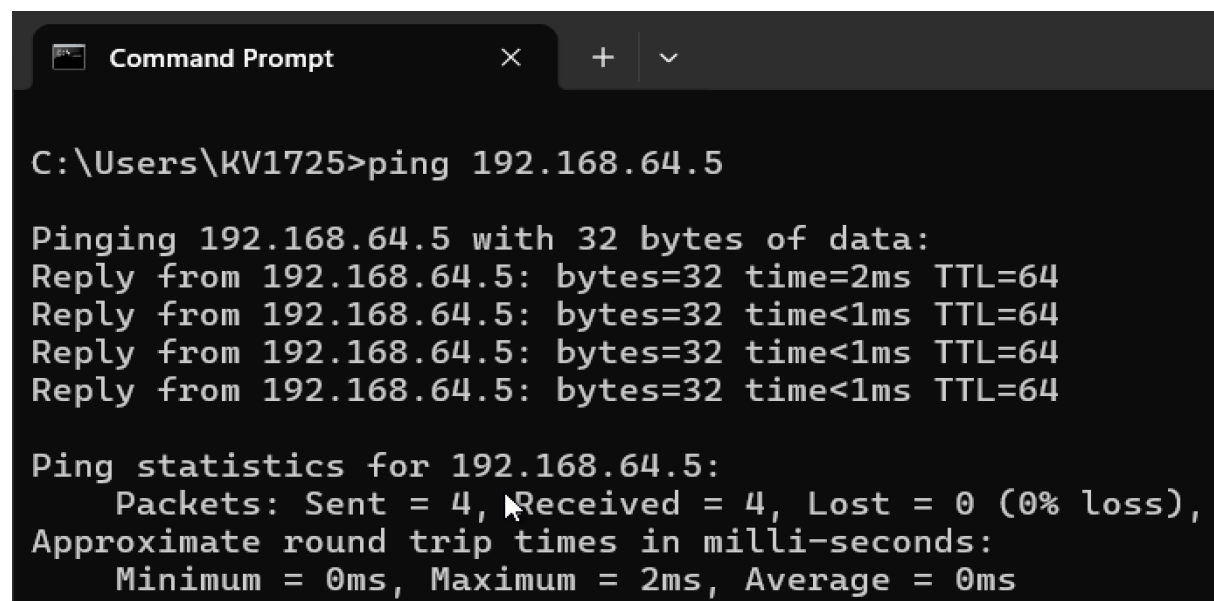
https://www.canva.com/design/DAHH_sGzIbc/m0Rk26o2FTAfHdKJjxU81g/watch

Github Link - <https://github.com/karnishwar1725>

8. Appendix:

```
ubuntu@ubuntu:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 1a:63:23:39:a4:2c brd ff:ff:ff:ff:ff:ff
    inet 192.168.64.5/24 metric 100 brd 192.168.64.255 scope global dynamic enp0s1
        valid_lft 3525sec preferred_lft 3525sec
    inet6 fd38:e520:ad90:2d1f:1863:23ff:fe39:a42c/64 scope global dynamic mngtmpaddr noprefixroute
        valid_lft 2591943sec preferred_lft 604743sec
    inet6 fe80::1863:23ff:fe39:a42c/64 scope link
        valid_lft forever preferred_lft forever
```

Image 1 - Ip of Honeygot and SIEM VM



The screenshot shows a Windows Command Prompt window with the title bar 'Command Prompt'. The command prompt shows the user is at 'C:\Users\KV1725' and has entered 'ping 192.168.64.5'. The output shows four successful replies from 192.168.64.5 with 32 bytes of data, times less than 1ms, and TTL=64. Below the replies, it shows 'Ping statistics for 192.168.64.5:' with 'Packets: Sent = 4, Received = 4, Lost = 0 (0% loss)', 'Approximate round trip times in milli-seconds:' and 'Minimum = 0ms, Maximum = 2ms, Average = 0ms'.

```
C:\Users\KV1725>ping 192.168.64.5

Pinging 192.168.64.5 with 32 bytes of data:
Reply from 192.168.64.5: bytes=32 time=2ms TTL=64
Reply from 192.168.64.5: bytes=32 time<1ms TTL=64
Reply from 192.168.64.5: bytes=32 time<1ms TTL=64
Reply from 192.168.64.5: bytes=32 time<1ms TTL=64

Ping statistics for 192.168.64.5:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 2ms, Average = 0ms
```

Image 2 - Attacker VM to Honeygot VM ping Success

```

ubuntu@ubuntu:~$ sudo apt install openssh-server -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages were automatically installed and are no longer required:
  libwpe-1.0-1 libwpebackend-fdo-1.0-1
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
  openssh-client openssh-sftp-server
Suggested packages:
  keychain libpam-ssh monkeysphere ssh-askpass molly-guard ufw
The following packages will be upgraded:
  openssh-client openssh-server openssh-sftp-server
3 upgraded, 0 newly installed, 0 to remove and 298 not upgraded.
Need to get 0 B/1309 kB of archives.
After this operation, 5120 B of additional disk space will be used.
debconf: delaying package configuration, since apt-utils is not installed
(Reading database ... 207109 files and directories currently installed.)
Preparing to unpack .../openssh-sftp-server_1%3a8.9p1-3ubuntu0.14_arm64.deb ...
Unpacking openssh-sftp-server (1:8.9p1-3ubuntu0.14) over (1:8.9p1-3ubuntu0.13) ...
Preparing to unpack .../openssh-server_1%3a8.9p1-3ubuntu0.14_arm64.deb ...
Unpacking openssh-server (1:8.9p1-3ubuntu0.14) over (1:8.9p1-3ubuntu0.13) ...
Preparing to unpack .../openssh-client_1%3a8.9p1-3ubuntu0.14_arm64.deb ...
Unpacking openssh-client (1:8.9p1-3ubuntu0.14) over (1:8.9p1-3ubuntu0.13) ...
Setting up openssh-client (1:8.9p1-3ubuntu0.14) ...
Setting up openssh-sftp-server (1:8.9p1-3ubuntu0.14) ...
Setting up openssh-server (1:8.9p1-3ubuntu0.14) ...
debconf: unable to initialize frontend: Dialog
debconf: (No usable dialog-like program is installed, so the dialog based frontend cannot be used. at /usr/share/perl5/Debconf/FrontEnd/Dialog.pm line 78.)
debconf: falling back to frontend: Readline
rescue-ssh.target is a disabled or a static unit not running, not starting it.
ssh.socket is a disabled or a static unit not running, not starting it.
Processing triggers for man-db (2.10.2-1) ...
debconf: unable to initialize frontend: Dialog
debconf: (No usable dialog-like program is installed, so the dialog based frontend cannot be used. at /usr/share/perl5/Debconf/FrontEnd/Dialog.pm line 78.)
debconf: falling back to frontend: Readline
Scanning processes...
Scanning candidates...
Scanning linux images...
Pending kernel upgrade

```

Image 3 - SSH Honeypot Installed

```

ubuntu@ubuntu:~$ sudo systemctl enable ssh
Synchronizing state of ssh.service with SysV service script with /lib/systemd/systemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install enable ssh
ubuntu@ubuntu:~$ sudo systemctl start ssh
ubuntu@ubuntu:~$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/lib/systemd/system/ssh.service; enabled; vendor preset: enabled)
   Active: active (running) since Sat 2026-04-25 18:42:56 UTC; 2min 56s ago
     Docs: man:sshd(8)
           man:sshd_config(5)
  Main PID: 89342 (sshd)
    Tasks: 1 (limit: 4518)
   Memory: 1.7M
      CPU: 26ms
   CGroup: /system.slice/ssh.service
           └─89342 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Apr 25 18:42:56 ubuntu systemd[1]: Starting OpenBSD Secure Shell server...
Apr 25 18:42:56 ubuntu sshd[89342]: Server listening on 0.0.0.0 port 22.
Apr 25 18:42:56 ubuntu sshd[89342]: Server listening on :: port 22.
Apr 25 18:42:56 ubuntu systemd[1]: Started OpenBSD Secure Shell server.

```

Image 4 - Honeypot Running

```
#LoginGraceTime 2m
PermitRootLogin yes
#StrictModes yes
#MaxAuthTries 6
#MaxSessions 10

#PubkeyAuthentication yes

# Expect .ssh/authorized_keys2 to be disregarded by default in future.
#AuthorizedKeysFile      .ssh/authorized_keys .ssh/authorized_keys2

#AuthorizedPrincipalsFile none

#AuthorizedKeysCommand none
#AuthorizedKeysCommandUser nobody

# For this to work you will also need host keys in /etc/ssh/ssh_known_hosts
#HostbasedAuthentication no
# Change to yes if you don't trust ~/.ssh/known_hosts for
# HostbasedAuthentication
#IgnoreUserKnownHosts no
# Don't read the user's ~/.rhosts and ~/.shosts files
#IgnoreRhosts yes

# To disable tunneled clear text passwords, change to no here!
PasswordAuthentication yes
#PermitEmptyPasswords no
```

Image 5 - Config file edited for Authentication

```
ubuntu@ubuntu:~$ sudo adduser admin
Adding user `admin' ...
Adding new group `admin' (1002) ...
Adding new user `admin' (1002) with group `admin' ...
Creating home directory `/home/admin' ...
Copying files from `/etc/skel' ...
New password:
BAD PASSWORD: The password contains the user name in some form
Retype new password:
passwd: password updated successfully
Changing the user information for admin
Enter the new value, or press ENTER for the default
    Full Name []: admin
    Room Number []:
    Work Phone []:
    Home Phone []:
    Other []:
Is the information correct? [Y/n] y
ubuntu@ubuntu:~$ sudo adduser test
Adding user `test' ...
Adding new group `test' (1003) ...
Adding new user `test' (1003) with group `test' ...
Creating home directory `/home/test' ...
Copying files from `/etc/skel' ...
New password:
BAD PASSWORD: The password fails the dictionary check - it is based on a dictionary word
Retype new password:
passwd: password updated successfully
Changing the user information for test
Enter the new value, or press ENTER for the default
    Full Name []: test
    Room Number []:
    Work Phone []:
    Home Phone []:
    Other []:
Is the information correct? [Y/n] y
ubuntu@ubuntu:~$
```

Image 6 - Fake Users Created

```
C:\Users\KV1725>ssh admin@192.168.64.5
admin@192.168.64.5's password:
Permission denied, please try again.
admin@192.168.64.5's password:
Permission denied, please try again.
admin@192.168.64.5's password:
admin@192.168.64.5: Permission denied (publickey,password).

C:\Users\KV1725>ssh admin@192.168.64.5
admin@192.168.64.5's password:
Permission denied, please try again.
admin@192.168.64.5's password:
Permission denied, please try again.
admin@192.168.64.5's password:
admin@192.168.64.5: Permission denied (publickey,password).

C:\Users\KV1725>ssh admin@192.168.64.5
admin@192.168.64.5's password:
Permission denied, please try again.
admin@192.168.64.5's password:
Permission denied, please try again.
admin@192.168.64.5's password:
admin@192.168.64.5: Permission denied (publickey,password).
```

Image 7 - Failed SSH Login Attempts

```
C:\Users\KV1725>ssh admin@192.168.64.5
admin@192.168.64.5's password:
Welcome to Ubuntu 22.04.3 LTS (GNU/Linux 5.15.0-87-generic aarch64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

Expanded Security Maintenance for Applications is not enabled.

307 updates can be applied immediately.
156 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

New release '24.04.4 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

*** System restart required ***
Last login: Sat Apr 25 18:59:03 2026 from 192.168.64.3
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

admin@ubuntu:~$ |
```

Image 8 - Successful SSH Login


```

admin@ubuntu:/$ whoami
admin
admin@ubuntu:/$ pwd
/
admin@ubuntu:/$ cd /home
admin@ubuntu:/home$ ls
admin  cowrie  test  ubuntu
admin@ubuntu:/home$ uname -a
Linux ubuntu 5.15.0-87-generic #97-Ubuntu SMP Tue Oct 3 09:52:42 UTC 2023 aarch64 aarch64 aarch64 GNU/Linux
admin@ubuntu:/home$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
_apt:x:100:65534::/nonexistent:/usr/sbin/nologin
systemd-network:x:101:102:systemd Network Management,,,:/run/systemd:/usr/sbin/nologin
systemd-resolve:x:102:103:systemd Resolver,,,:/run/systemd:/usr/sbin/nologin
messagebus:x:103:104::/nonexistent:/usr/sbin/nologin
systemd-timesync:x:104:105:systemd Time Synchronization,,,:/run/systemd:/usr/sbin/nologin
pollinate:x:105:1::/var/cache/pollinate:/bin/false
sshd:x:106:65534::/run/sshd:/usr/sbin/nologin
ubuntu:x:1000:1000:ubuntu:/home/ubuntu:/bin/bash
tss:x:107:112:TPM software stack,,,:/var/lib/tpm:/bin/false
rtkit:x:108:113:RealtimeKit,,,:/proc:/usr/sbin/nologin
kernoops:x:109:65534:Kernel Oops Tracking Daemon,,,:/usr/sbin/nologin
systemd-oom:x:110:117:systemd Userspace OOM Killer,,,:/run/systemd:/usr/sbin/nologin
whoopsie:x:111:118::/nonexistent:/bin/false
avahi-autoipd:x:112:119:Avahi autoip daemon,,,:/var/lib/avahi-autoipd:/usr/sbin/nologin
usbmux:x:113:46:usbmux daemon,,,:/var/lib/usbmux:/usr/sbin/nologin
nm-openvpn:x:114:120:NetworkManager OpenVPN,,,:/var/lib/openvpn/chroot:/usr/sbin/nologin
dnsmasq:x:115:65534:dnsmasq,,,:/var/lib/misc:/usr/sbin/nologin
avahi:x:116:122:Avahi mDNS daemon,,,:/run/avahi-daemon:/usr/sbin/nologin
cups-pk-helper:x:117:123:user for cups-pk-helper service,,,:/home/cups-pk-helper:/usr/sbin/nologin
sssd:x:118:124:SSSD system user,,,:/var/lib/sss:/usr/sbin/nologin
speech-dispatcher:x:119:29:Speech Dispatcher,,,:/run/speech-dispatcher:/bin/false
fwupd-refresh:x:120:125:fwupd-refresh user,,,:/run/systemd:/usr/sbin/nologin
saned:x:121:127::/var/lib/saned:/usr/sbin/nologin
colord:x:122:128:colord colour management daemon,,,:/var/lib/colord:/usr/sbin/nologin
geoclue:x:123:129::/var/lib/geoclue:/usr/sbin/nologin
pulse:x:124:130:PulseAudio daemon,,,:/run/pulse:/usr/sbin/nologin
gnome-initial-setup:x:125:65534::/run/gnome-initial-setup:/bin/false
hplip:x:126:7:HPLIP system user,,,:/run/hplip:/bin/false
gdm:x:127:132:Gnome Display Manager:/var/lib/gdm3:/bin/false
cowrie:x:1001:1001:Honeypot,,,:/home/cowrie:/bin/bash
admin:x:1002:1002:admin,,,:/home/admin:/bin/bash
test:x:1003:1003:test,,,:/home/test:/bin/bash
admin@ubuntu:/home$

```

Image 9 - Commands executed by attacker

```

ubuntu@ubuntu:~$ sudo journalctl -u ssh -f
Apr 25 19:00:35 ubuntu sshd[90961]: Connection reset by authenticating user admin 192.168.64.3 port 55104 [preauth]
Apr 25 19:00:35 ubuntu sshd[90961]: PAM 2 more authentication failures; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.168.64.3 user=admin
Apr 25 19:00:46 ubuntu sshd[90963]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.168.64.3 user=admin
Apr 25 19:00:49 ubuntu sshd[90963]: Failed password for admin from 192.168.64.3 port 55105 ssh2
Apr 25 19:00:54 ubuntu sshd[90963]: Failed password for admin from 192.168.64.3 port 55105 ssh2
Apr 25 19:00:56 ubuntu sshd[90963]: Failed password for admin from 192.168.64.3 port 55105 ssh2
Apr 25 19:00:56 ubuntu sshd[90963]: Connection reset by authenticating user admin 192.168.64.3 port 55105 [preauth]
Apr 25 19:00:56 ubuntu sshd[90963]: PAM 1 more authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.168.64.3 user=admin
Apr 25 19:01:35 ubuntu sshd[90965]: Accepted password for admin from 192.168.64.3 port 53349 ssh2
Apr 25 19:01:35 ubuntu sshd[90965]: pam_unix(sshd:session): session opened for user admin(uid=1002) by (uid=0)

```

Image 10 - Honeypot logs

```

ubuntu@ubuntu:~$ ls /var/log/journal
4fb1781ac9f64d5cad47e77f1ed4f268
ubuntu@ubuntu:~$ sudo journalctl -u ssh | head
Oct 21 00:06:21 ubuntu systemd[1]: Starting OpenBSD Secure Shell server...
Oct 21 00:06:21 ubuntu sshd[824]: Server listening on 0.0.0.0 port 22.
Oct 21 00:06:21 ubuntu sshd[824]: Server listening on :: port 22.
Oct 21 00:06:21 ubuntu systemd[1]: Started OpenBSD Secure Shell server.
Oct 21 00:20:10 ubuntu sshd[824]: Received signal 15; terminating.
Oct 21 00:20:10 ubuntu systemd[1]: Stopping OpenBSD Secure Shell server...
Oct 21 00:20:10 ubuntu systemd[1]: ssh.service: Deactivated successfully.
Oct 21 00:20:10 ubuntu systemd[1]: Stopped OpenBSD Secure Shell server.
Oct 21 00:20:10 ubuntu systemd[1]: Starting OpenBSD Secure Shell server...
Oct 21 00:20:10 ubuntu sshd[38798]: Server listening on 0.0.0.0 port 22.

```

Image 11 - Log file exists and Logs are stored

```

ubuntu@ubuntu:~$ sudo systemctl daemon-reexec
ubuntu@ubuntu:~$ sudo systemctl enable elasticsearch
Created symlink /etc/systemd/system/multi-user.target.wants/elasticsearch.service → /lib/systemd/system/elasticsearch.service.
ubuntu@ubuntu:~$ sudo systemctl start elasticsearch
ubuntu@ubuntu:~$ sudo systemctl status elasticsearch
● elasticsearch.service - Elasticsearch
   Loaded: loaded (/lib/systemd/system/elasticsearch.service; enabled; vendor preset: enabled)
   Active: active (running) since Sat 2026-04-25 20:40:28 UTC; 3s ago
     Docs: https://www.elastic.co
   Main PID: 95325 (java)
    Tasks: 101 (limit: 4518)
   Memory: 2.2G
      CPU: 25.189s
   CGroup: /system.slice/elasticsearch.service
           └─95325 /usr/share/elasticsearch/jdk/bin/java -Xms4m -Xmx64m -XX:+UseSerialGC -Dcli.name=server -Dcli.script=/usr/share/e
             └─95387 /usr/share/elasticsearch/jdk/bin/java -Des.networkaddress.cache.ttl=60 -Des.networkaddress.cache.negative.ttl=10
               └─95410 /usr/share/elasticsearch/modules/x-pack-m/ml/platform/linux-aarch64/bin/controller

Apr 25 20:40:11 ubuntu systemd[1]: Starting Elasticsearch...
Apr 25 20:40:28 ubuntu systemd[1]: Started Elasticsearch.
lines 1-15/15 (END)...skipping...
● elasticsearch.service - Elasticsearch
   Loaded: loaded (/lib/systemd/system/elasticsearch.service; enabled; vendor preset: enabled)
   Active: active (running) since Sat 2026-04-25 20:40:28 UTC; 3s ago
     Docs: https://www.elastic.co
   Main PID: 95325 (java)
    Tasks: 101 (limit: 4518)
   Memory: 2.2G
      CPU: 25.189s
   CGroup: /system.slice/elasticsearch.service
           └─95325 /usr/share/elasticsearch/jdk/bin/java -Xms4m -Xmx64m -XX:+UseSerialGC -Dcli.name=server -Dcli.script=/usr/share/e
             └─95387 /usr/share/elasticsearch/jdk/bin/java -Des.networkaddress.cache.ttl=60 -Des.networkaddress.cache.negative.ttl=10
               └─95410 /usr/share/elasticsearch/modules/x-pack-m/ml/platform/linux-aarch64/bin/controller

Apr 25 20:40:11 ubuntu systemd[1]: Starting Elasticsearch...
Apr 25 20:40:28 ubuntu systemd[1]: Started Elasticsearch.

```

Image 12 - Elastic Search active and running

```
ubuntu@ubuntu:~$ sudo systemctl enable kibana
Created symlink /etc/systemd/system/multi-user.target.wants/kibana.service → /lib/systemd/system/kibana.service.
ubuntu@ubuntu:~$ sudo systemctl start kibana
ubuntu@ubuntu:~$ sudo systemctl status kibana
● kibana.service - Kibana
   Loaded: loaded (/lib/systemd/system/kibana.service; enabled; vendor preset: enabled)
   Active: active (running) since Sat 2026-04-25 20:43:54 UTC; 10s ago
     Docs: https://www.elastic.co
   Main PID: 96218 (node)
    Tasks: 11 (limit: 4518)
   Memory: 434.5M
      CPU: 5.101s
   CGroup: /system.slice/kibana.service
           └─96218 /usr/share/kibana/bin/../node/glibc-217/bin/node /usr/share/kibana/bin/../src/cli/kibana/dist

Apr 25 20:43:54 ubuntu kibana[96218]: Kibana is currently running with legacy OpenSSL providers enabled! For details and instructions>
Apr 25 20:43:54 ubuntu kibana[96218]: {"log.level":"info","@timestamp":"2026-04-25T20:43:54.711Z","log.logger":"elastic-apm-node","ec>
Apr 25 20:43:54 ubuntu kibana[96218]: Native global console methods have been overridden in production environment.
Apr 25 20:43:55 ubuntu kibana[96218]: [2026-04-25T20:43:55.302+00:00][INFO ][root] Kibana is starting
Apr 25 20:43:55 ubuntu kibana[96218]: [2026-04-25T20:43:55.323+00:00][INFO ][node] Kibana process configured with roles: [background>
Apr 25 20:43:58 ubuntu kibana[96218]: [2026-04-25T20:43:58.516+00:00][INFO ][plugins-service] The following plugins are disabled: "c>
Apr 25 20:43:58 ubuntu kibana[96218]: [2026-04-25T20:43:58.546+00:00][INFO ][[http.server.Preboot] http server running at http://local>
Apr 25 20:43:58 ubuntu kibana[96218]: [2026-04-25T20:43:58.587+00:00][INFO ][plugins-system.preboot] Setting up [1] plugins: [interac>
Apr 25 20:43:58 ubuntu kibana[96218]: [2026-04-25T20:43:58.595+00:00][INFO ][preboot] "interactiveSetup" plugin is holding setup: Val>
Apr 25 20:43:58 ubuntu kibana[96218]: [2026-04-25T20:43:58.608+00:00][INFO ][root] Holding setup until preboot stage is completed.
lines 1-21/21 (END)
```

Image 13 - Kibana Installed and Running

```
# ===== System: Kibana Server =====
# Kibana is served by a back end server. This setting specifies the port to use.
#server.port: 5601

# Specifies the address to which the Kibana server will bind. IP addresses and host names are both valid values.
# The default is 'localhost', which usually means remote machines will not be able to connect.
# To allow connections from remote users, set this parameter to a non-loopback address.
server.host: "0.0.0.0"
```

Image 14 - Kibana Config File Edited

```
ubuntu@ubuntu:~$ sudo /usr/share/elasticsearch/bin/elasticsearch-create-enrollment-token -s kibana  
eyJ2ZXIiOiI4LjE0LjAiLCJhZHIiOiIsImRlcjE2OjC4MzQ1OjkYMDAixSwiZmdyIjoiodQ0ZTQzMThkMGVlbnJDI4ZTgwWWE5Yzg4ZWRRjNjhincVlZjk1ZDRNGU1NGVMYz  
M1NmY0VWYkYjgXZGFLSIsIntleSI6Ii00aHB4cDBCMG5hUEp4d2lhZ0VNOjNSckBScm9xM3FMcjFfMFUHR1RlElfq==  
ubuntu@ubuntu:~$
```

Image 15 - Kibana enrolment Token Generated

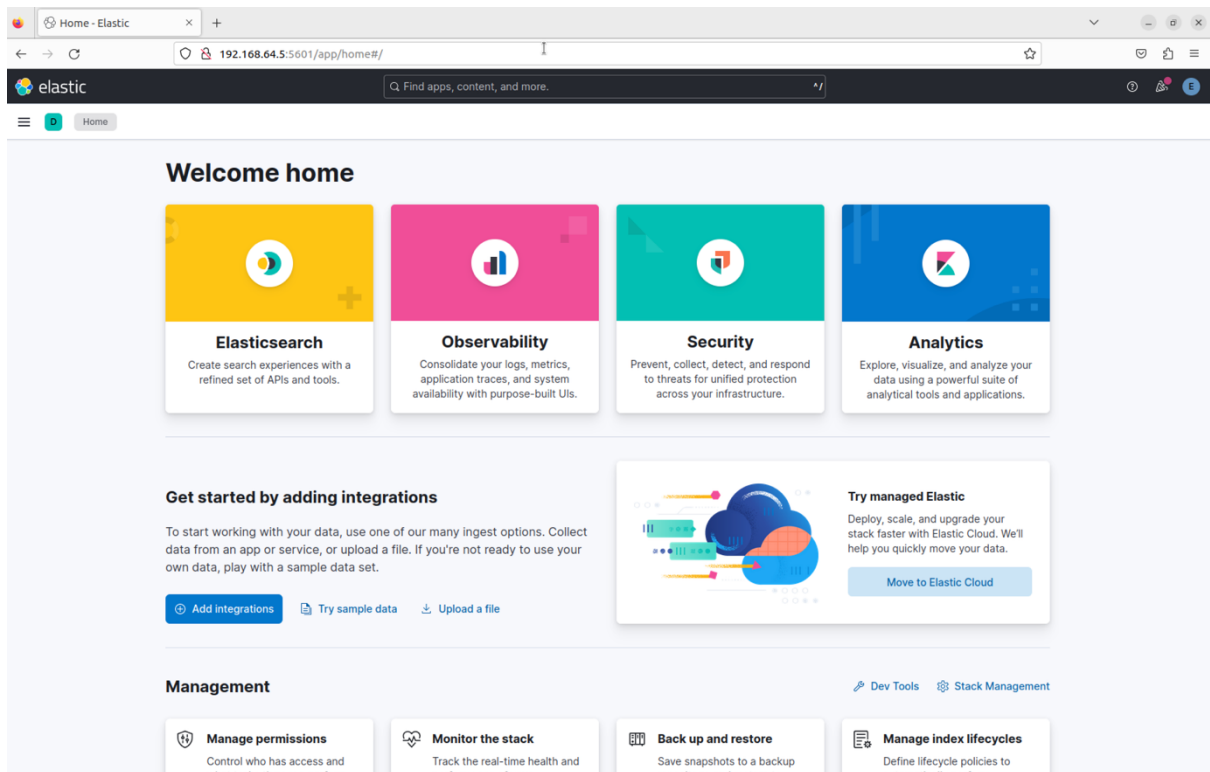


Image 16 - Elastic Dashboard

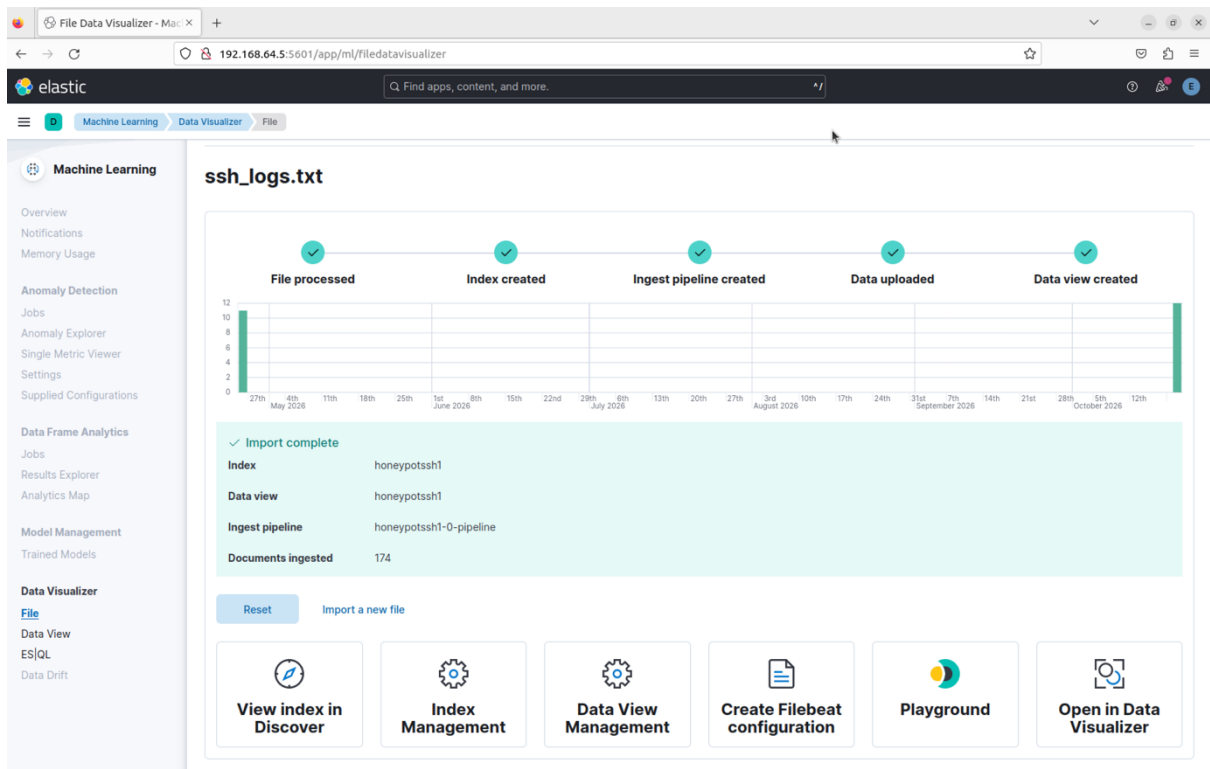


Image 17 - Log File Uploaded

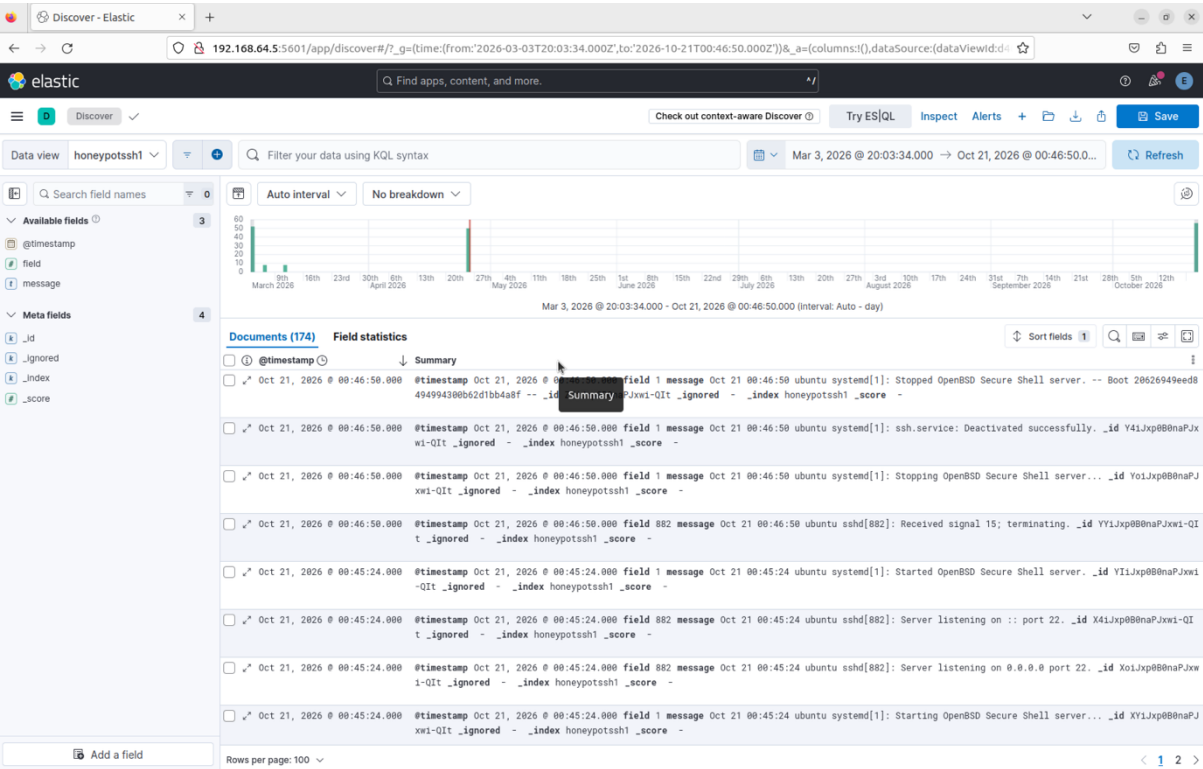


Image 18 - Log Analysis Evidence Table View

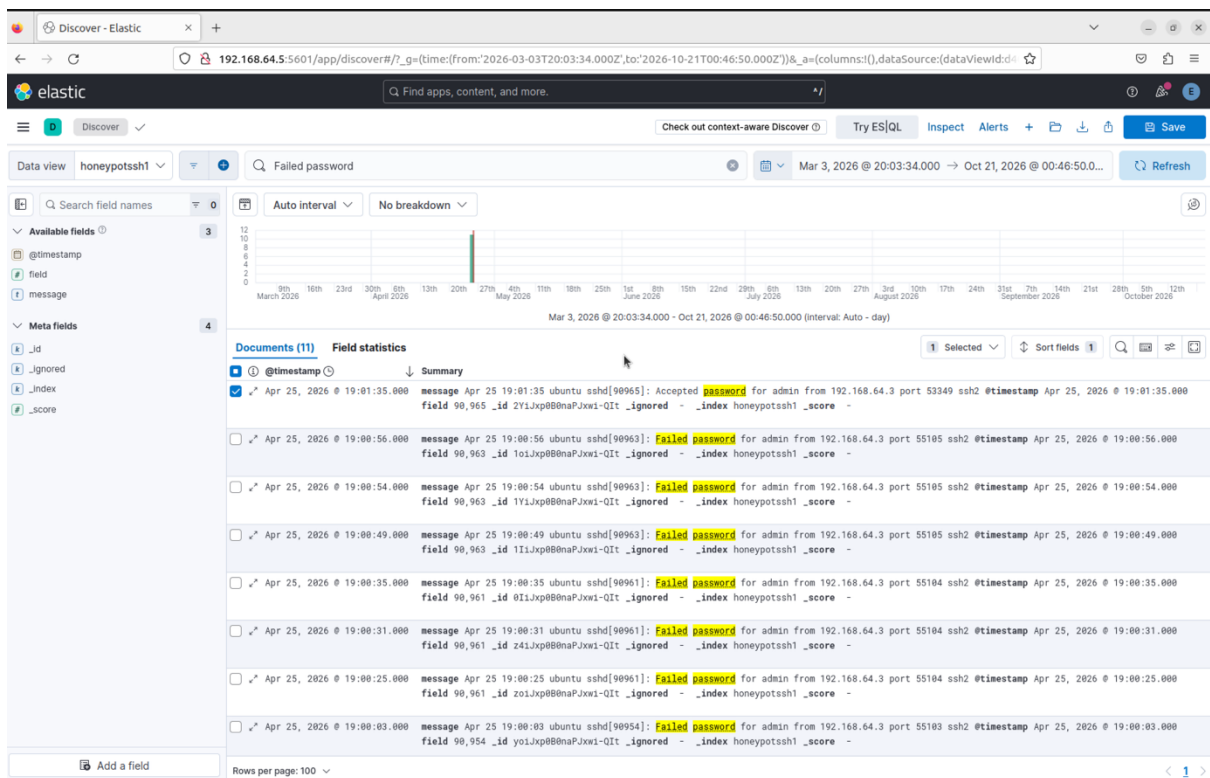


Image 19 - Multiple Failed Login attempts

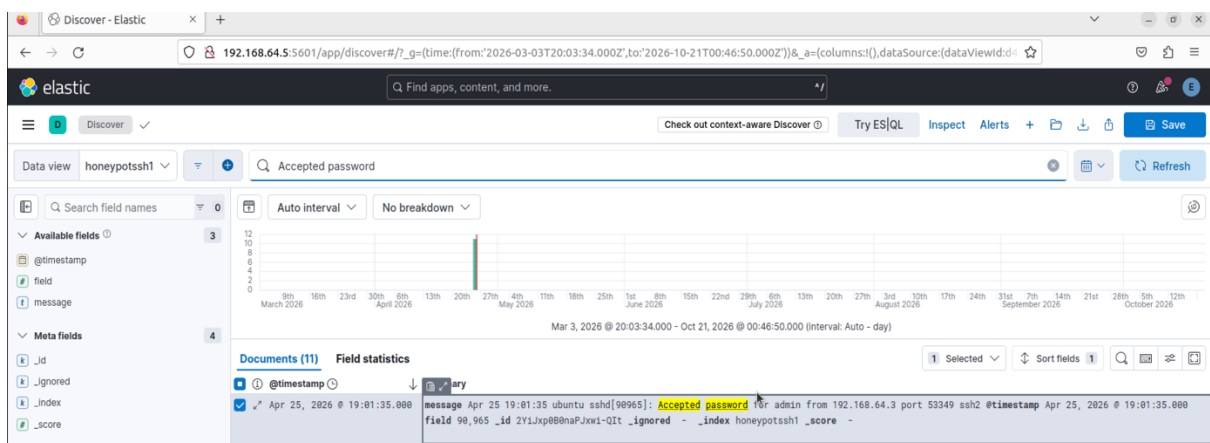


Image 20 - Successful Login Attempt

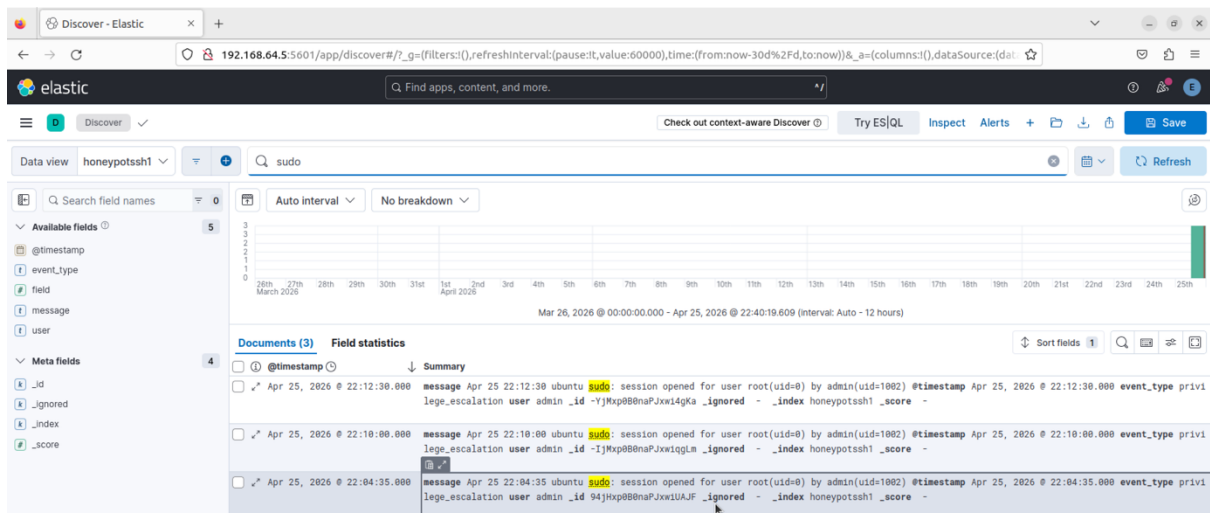


Image 21 - Sudo and Privilege Escalation

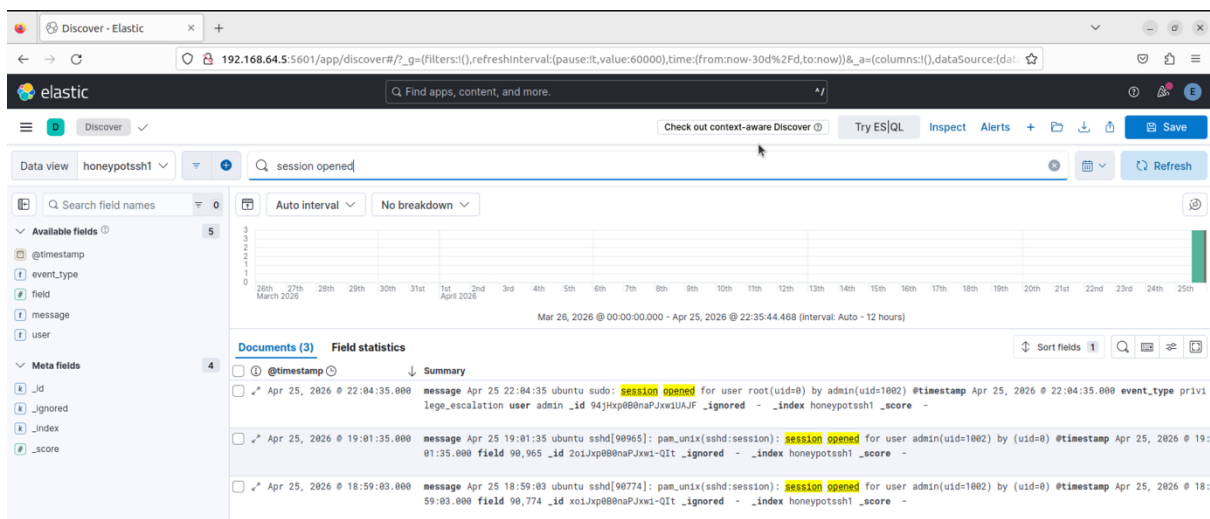


Image 22 - Session Opened

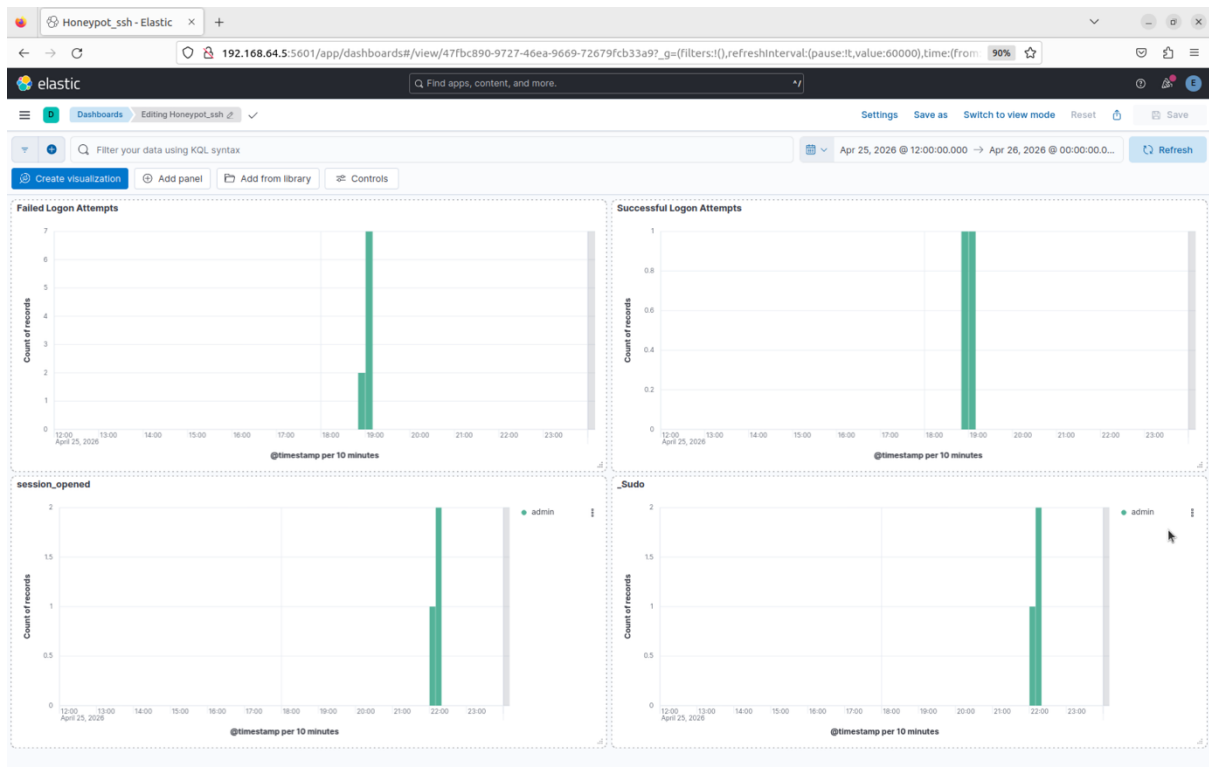


Image 23 – Visualization

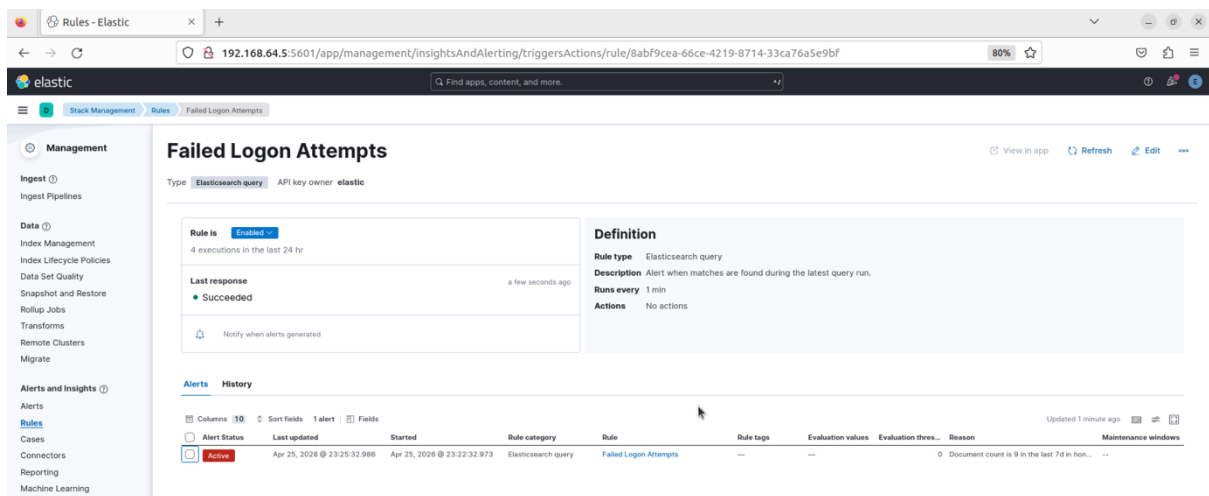


Image 24 - Failed Logon Attempts Rule Triggered

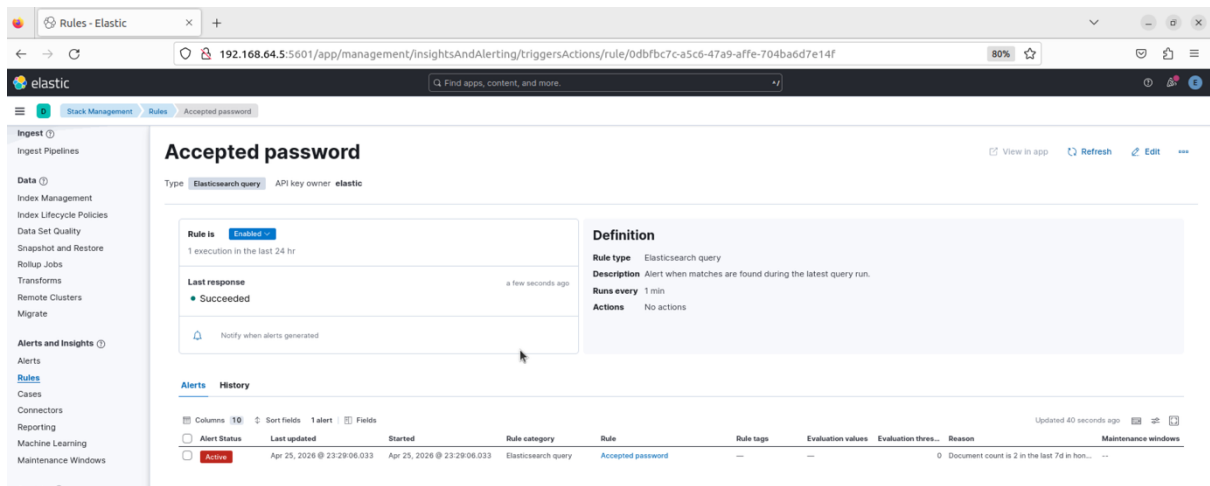


Image 25 - Successful Logon Attempts Rule Triggered

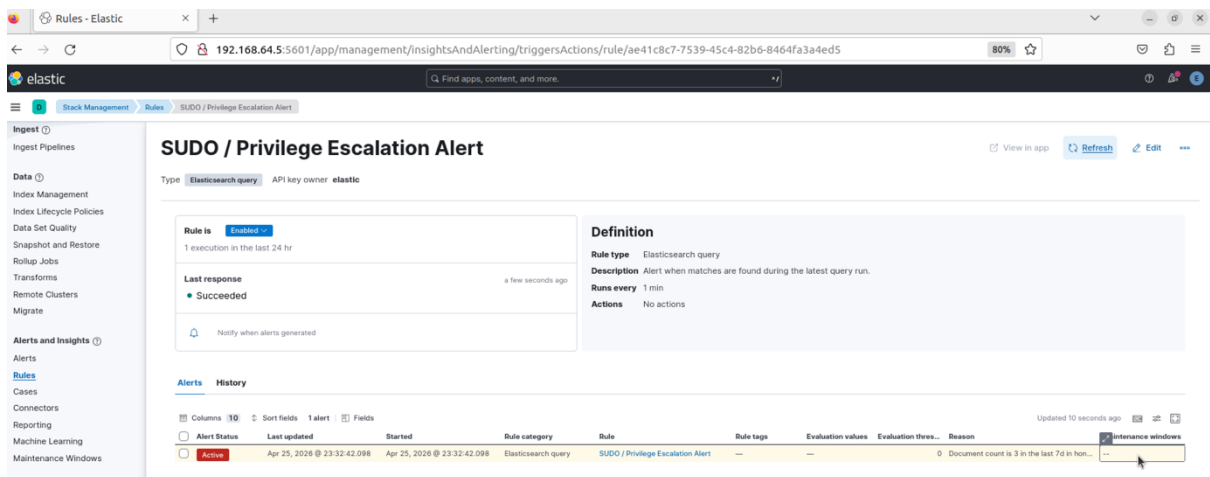


Image 26 - SUDO/ Privilege Esc Attempts Rule Triggered

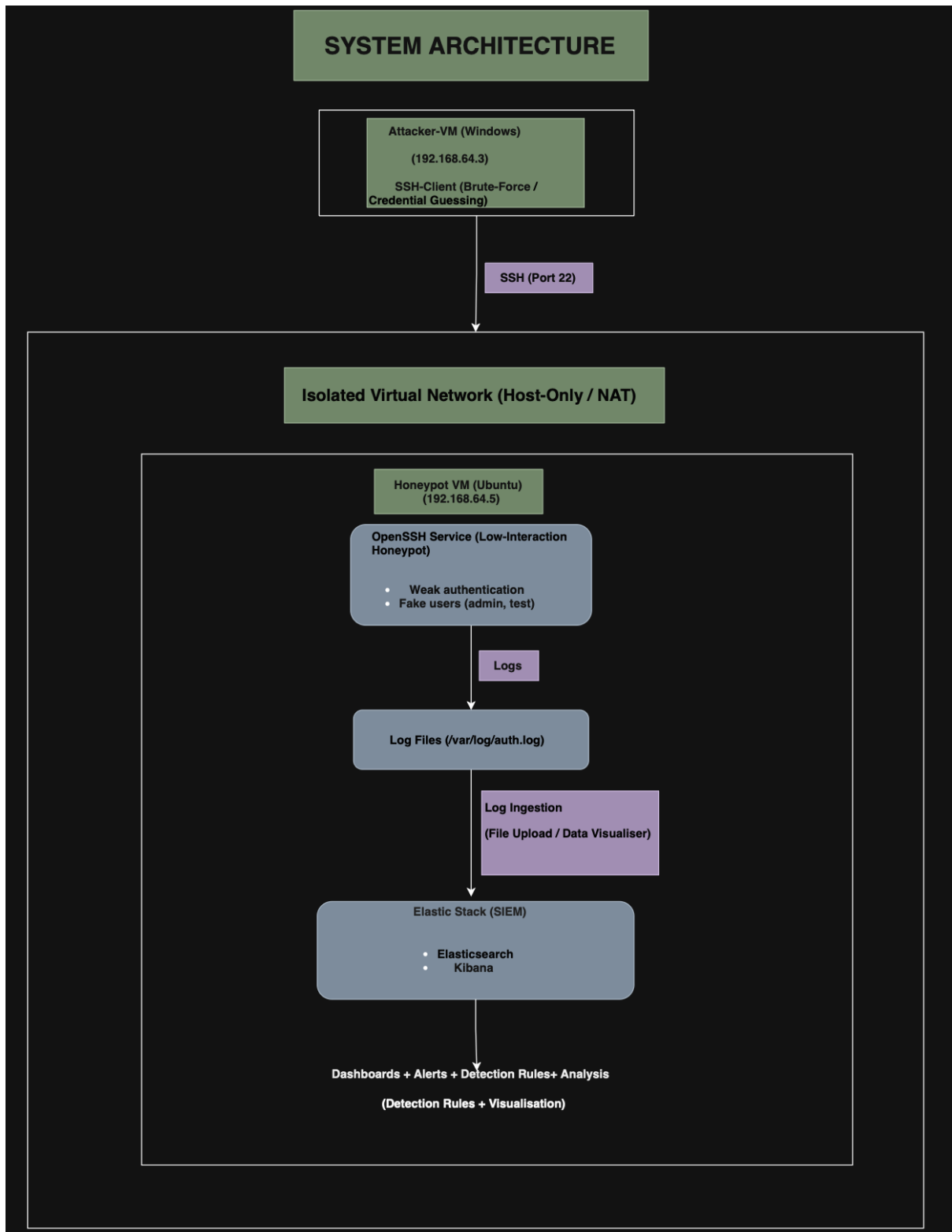


Image 27 – System Architecture Diagram